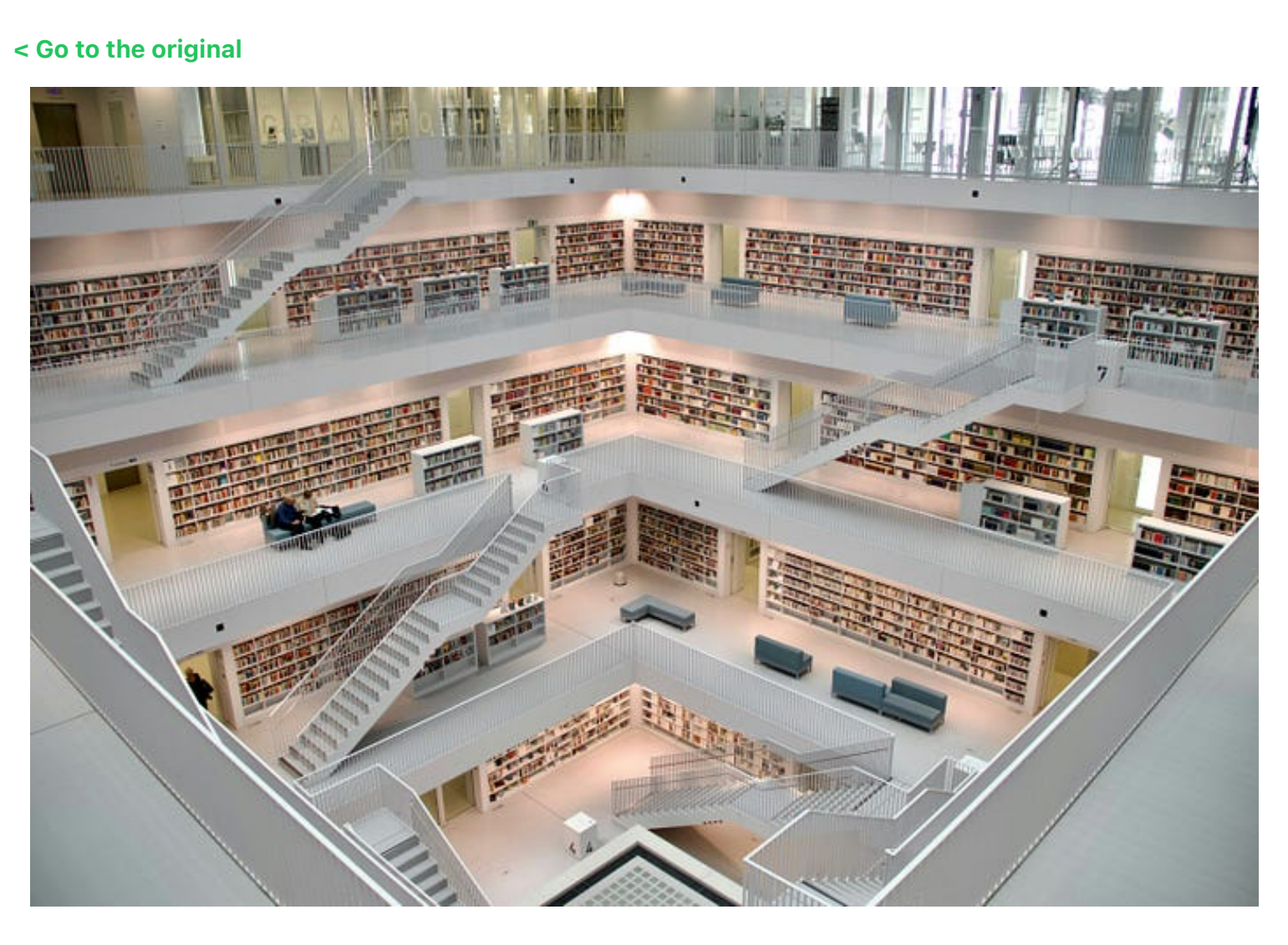
< Go to the original



How to Build a Knowledge Graph in Minutes (And Make It Enterprise-Ready)

I tried and failed creating one—but it was when LLMs were not a thing!

Thuwarakesh Murallie
Follow

Towards Data Science · a11y-light · ~8 min read · January 13, 2025 (Updated: January 13, 2025) ·
Free: No

At first, knowledge graphs (KGs) sounded daunting—not the concept, but the process of constructing one.

I've tried constructing a knowledge graph before and failed.

Graphs are undoubtedly one of the best ways to represent complex relationships. They have many uses, such as recommendation systems and fraud detection. However, the one that caught my interest the most was information retrieval.

I started using knowledge graphs to build better RAGs.

RAGs don't necessarily demand knowledge graphs. They don't demand a database at all. As long as you can extract relevant information from a large pool and pass it to the context of an LLM, RAGs work.

You could build a RAG with a web search as its information retrieval strategy or use a vector store to benefit from its semantic text search features.

If you use a graph database to retrieve contextual information, we call it GraphRAG.

This isn't a post about GraphRAGs (Perhaps in a future post). This is about the construction of knowledge graphs itself using LLMs. But it's worth mentioning how KG as a content store improves RAGs. So here it is.

Why Knowledge Graphs for RAG?

Knowledge graphs offer clever techniques for retrieving more relevant information. Vector stores, although they usually serve their purpose, sometimes fall short.

The primary retrieval technique from vector stores is the semantic similarity of encoded text. Here's how it works.

We use a vector embedding model, such as OpenAI's text-embedding-3, to create a vector representation of the text. This makes Apple and Appam (an Indian food) very different in their vector representation, even though they share many standard letters. These vectors are then stored in a vector database like Chroma.

During the retrieval phase, we encode the user input using the same embedding model we used. We can then retrieve information from the vector store using distance matrices, such as cosine similarity.

This approach is the only way we retrieve information from vector stores. As you may have already guessed, the embedding model we use plays a key role in the accuracy of the retrieval process. The type of database is rarely an issue as far as the accuracy is concerned (, but there are issues like concurrency, speed, etc.)

Here's an example:

Imagine you've got a massive document about the leadership team of various companies.

A vector embedding system effectively handles simple factual questions like "*What organization does Mr. John Doe serve as CEO?*" because the answer is directly represented in embedded document chunks.

However, broader or analytical queries, such as "*Who sits on more than one board with Mr. John Doe as a director?*" are challenging.

> Vector similarity searches rely on explicit mentions in the knowledge base. Knowledge graphs allow reasoning at a global dataset level.

Nearest neighbor searches rely on explicit mentions in the knowledge base. Without such summaries, embedding-based systems struggle to reason across multiple sources or synthesize information.

Knowledge graphs allow reasoning at a global dataset level. They could have country nodes and strategy nodes clustered closer. And we can run a simple query to get the information we want.

Now that we understand why knowledge graphs are important let's talk about the challenges of constructing one.

Constructing knowledge graphs was too tricky (In the past.)

A coworker introduced me to knowledge graphs a few years ago. He wants to create a unified, searchable KG for all our projects.

After a weekend learning Neo4J, it looked more promising.

The missing piece was extracting nodes and edges (entities and relationships) from an extensive collection of PDFs, PowerPoint slides, and Word documents.

The answer wasn't fruitful because we could only manually transfer the unstructured documents to the graph data model.

We could use PyPDF2 to read the PDFs and perform a keyword search for known nodes and edges. However, this method wasn't very successful, so we had to abandon the idea and mark it as NOT WORTH THE EFFORT.

This has changed as LLMs have become part of our day-to-day lives.

In the next section, we'll build a small (perhaps the simplest possible) knowledge graph with the help of LLM.

Building a knowledge graph in minutes

Unlike the old days, extracting information from text and images isn't that challenging.

I agree that handling unstructured data needs improvement. However, developments in the past few years, primarily using LLMs, have opened new possibilities.

This section will examine a knowledge graph construction process using LLMs and discuss how to improve it and make it enterprise-ready.

For this, we will use an experimental feature from Langchain called LLMGraphTransformer. We also use Neo4J Auro, a cloud-hosted solution, as the graph data store.

If you're using LlamaIndex, check out KnowledgeGraphIndex, an API similar to what we will use here. You can also use various other graph databases instead of Neo4J.

Let's start by installing the required packages.

Copy

```
pip install neo4j langchain-openai langchain-community langchain-experimental
```

In this example, we will map a list of business leaders, the organizations they are affiliated with, etc., to a graph database. If you'd like to follow along, find the sample data I used here. It's a dummy dataset I created (Of course, using AI.)

The surprisingly simple code that creates knowledge graphs from unstructured documents is as follows:

Copy

```
import os

from langchain_neo4j import Neo4jGraph
from langchain_openai import ChatOpenAI
from langchain_community.document_loaders import TextLoader
from langchain_experimental.graph_transformers import LLMGraphTransformer

graph = Neo4jGraph(
    url=os.getenv("NEO4J_URL"),
    username=os.getenv("NEO4J_USERNAME", "neo4j"),
    password=os.getenv("NEO4J_PASSWORD"),
)

# ------------------- Important --------------------
llm_transformer = LLMGraphTransformer(
    llm= ChatOpenAI(temperature=0, model_name="gpt-4-turbo")
)
# ------------------- Important --------------------

document = TextLoader("data/sample.txt").load()

graph_documents = llm_transformer.convert_to_graph_documents(document)

graph.add_graph_documents(graph_documents)
```

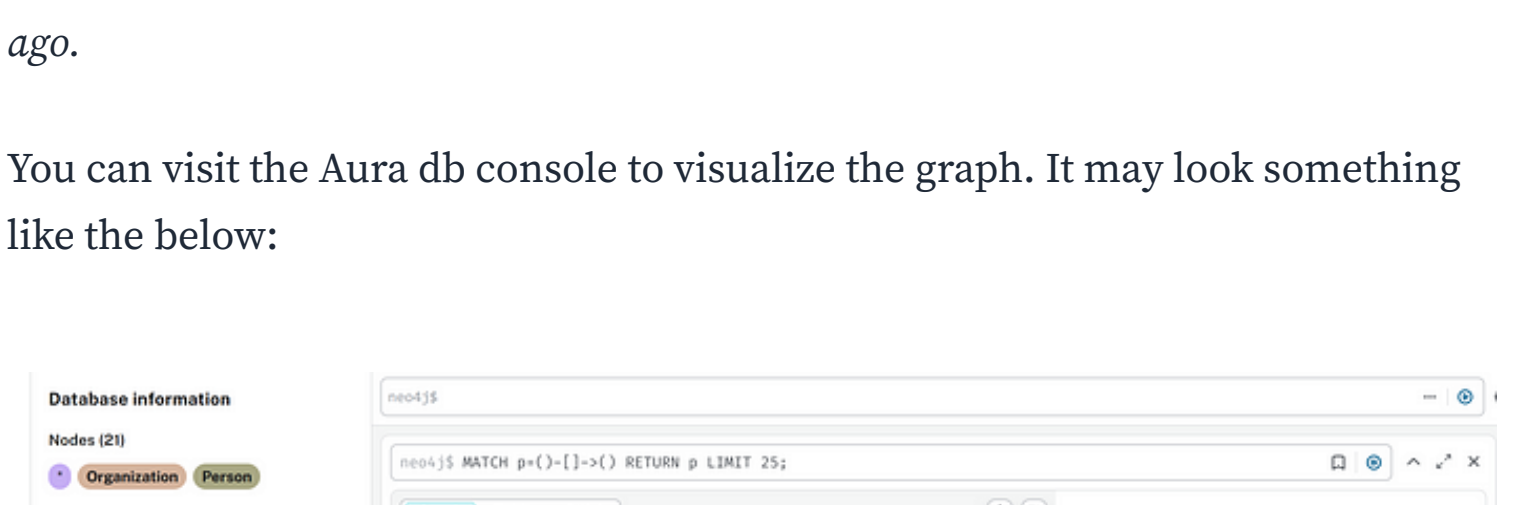
The code above is very straightforward.

I've highlighted the most crucial part of the code: constructing the graph database. The LLMGraphTransformer class uses the LLM we pass to it and extracts graphs from documents.

You can now pass in any Langchain Document type to the `convert_to_graph_documents` method to extract a knowledge graph. The source file can be a text, a markdown, a webpage, a response from another database query, etc.

If this had been a manual task, it would have taken months, as it was just a few years ago.

You can visit the Aura db console to visualize the graph. It may look something like the below:



```
neo4j$ MATCH p=()-[]->() RETURN p LIMIT 25;
```

Started streaming 22 records after 15 ms and completed after 21 ms.

Graph generated on Neo4J Aura cloud by LLMGraphTransformer module — Screenshot by the author.

Under the hood, the API uses an LLM to extract relevant information and constructs Neo4J Python objects for nodes and edges.

As we now know, the construction of knowledge graphs is effortless with the extractor API; let's discuss what we can do to make it usable for enterprise-grade applications.

How to make knowledge graphs enterprise-ready

We dropped the idea of constructing a knowledge graph because it was too complex. Indeed, we can now construct KGs faster with significantly less effort. However, the one we just created isn't ready for reliable applications.

I've identified a few flaws, which I also worked around. Two of them are worth discussing here.

1. Get more control over the graph extraction process

If you're following the example, you'd have noticed that the graph generated has only Person and Organization type nodes. Essentially, the extraction worked only for the people and the company they've been serving as directors.

Note: Using LLM to extract graphs is a probabilistic technique. Your outcome may not be the same as mine.

However, we can extract more information from the text file. For instance, we can determine which universities each executive attended and their prior experiences.

Could we specify the types of entities and relationships we need to extract? Fortunately, the LLMGraphTransformer class supports this.

Here's an alternative way to initiate the instance:

Copy

```
llm_transformer = LLMGraphTransformer(
    llm=llm,
    allowed_nodes=["Person", "Company", "University"],
    allowed_relationships=[
        ("Person", "CEO_OF", "Company"),
        ("Person", "CFO_OF", "Company"),
        ("Person", "CTO_OF", "Company"),
        ("Person", "STUDIED_AT", "University"),
    ],
    node_properties=True,
)
```

In the above version, we've explicitly told the transformer to look for Person, Company, and University type entities. And we've also educated the process on the relationships we can have between the entities. This is a vital piece of information for the LLM to extract entities.

Note that the third argument `node_properties` helps the transformer gather all the properties of the entities that aren't in a relationship with one another.

The knowledge graphs extracted by explicitly specifying the nodes and relationships are often more complete and accurate. But we can guarantee that all the key pieces of information are extracted well. For this, the following technique might help.

2. Propositioning before graph conversion

Text is a wired data form. And the brains that write these texts are even weirder.

Sometimes, we don't talk about everything straightforward. Even in formal and technical writing, we speak of the same thing in different places. Notice that in this post, I've used the words "Knowledge Graph" and "KG" in several places.

This makes LLMs interpret the context difficult.

Under the hood, the graph transformer still chunks the text and works independently within each chunk. Therefore, references to other chunks don't count.

For example, at the beginning of the text, you mentioned someone's position as Chief Investment Officer (CIO); however, during text splitting, the definition is lost in subsequent chunks. In the latter chunk, the LLM has no idea if the I in CIO stands for investment, information, or something else.

To tackle this, we use propositioning. Before chunking or processing any other text data, we proposition it so that every chunk is interpretable correctly.

I've spoken about propositioning a few times in some of my articles.

How to Achieve Near Human-Level Performance in Chunking for RAGs

The costly yet powerful splitting technique for superior RAG retrieval

towardsdatascience.com

Nonetheless, here's how to do it.

Copy

```
obj = hub.pull("wfh/proposal-indexing")

# You can explore the prompt template behind this by running the following:
# obj.get_prompts()[0].messages[0].prompt.template

llm = ChatOpenAI(model="gpt-4o")


# A Pydantic model to extract sentences from the passage
class Sentences(BaseModel):
    sentences: List[str]

extraction_llm = llm.with_structured_output(Sentences)


# Create the sentence extraction chain
extraction_chain = obj | extraction_llm


# Test it out
sentences = extraction_chain.invoke(
    """
    On July 20, 1969, astronaut Neil Armstrong walked on the moon .
    He was leading the NASA's Apollo 11 mission.
    Armstrong famously said, "That's one small step for man, one giant leap
    """
)



>>['On July 20, 1969, astronaut Neil Armstrong walked on the moon.',
 "Neil Armstrong was leading NASA's Apollo 11 mission.",
 'Neil Armstrong famously said, "That\'s one small step for man, one giant l
```

This code uses a prompt from Langchain's prompt hub. As you can see in the results, each chunk is self-explanable, even without any reference to the other chunks.

Doing this before creating a graph helps the LLM avoid losing nodes or relationships due to lost references.

Final thought

I've tried constructing a knowledge graph before and failed. The benefits of having one for an organization were far less than the effort required. But this was before LLMs were on the scene.

I was impressed when I learned that LLMs can extract graph information from plain text and put it in a database like Neo4J. But the experimental features aren't perfect.

I don't think they are production-ready — Not without some additional tweaks.

In this article, I listed two techniques that helped me improve the quality of the knowledge graph I constructed.

*Thanks for reading, friend! Besides **Medium**, I'm on **LinkedIn** and **X**, too!*

#retrieval-augmented #knowledge-graph #llm #artificial-intelligence #data-science